Detecting Wi-Fi Deauthentication Floods with Machine Learning

A local wireless intrusion detection system (WIDS) using a Random Forest classifier and a live dashboard

YOUR NAME
Student No. 000000
Course name and code
University / Department

The problem

Wi-Fi management messages are not authenticated. An attacker can forge **deauthentication** messages and repeatedly disconnect any device from its network. WPA3 and 802.11w help, but many networks still lack them.

How the system works

Traffic source: dataset, pcap or live adapter

↓

Events: device, time, connect or disconnect

↓

One-second windows per device

↓

7 timing features per window

↓

Random Forest (100 trees)

↓

Alert, alerts.csv and dashboard

The 7 features

Events / second	activity of one device
Disconnects / second	deauth + disassoc count
Disconnect share	disconnects / events
Shortest gap	attacks: milliseconds
Events, last 10 s	short burst
Events, last 60 s	longer activity
Time since last event	normal: long quiet

The data

376,429 association events

16 devices

149,008 device-seconds

16,596 attack seconds

Key finding: a normal device connects or disconnects at most once per second (about once every 105 s on average). An attacking device sends up to **335 events in one second**.

Normal device — 1 event / second max

Under attack — 335 events / second max

The model

A **Random Forest** is 100 decision trees that each look at the 7 features and vote. It is a ready-made scikit-learn classifier that labels each device-second as normal or malicious.

- Class-balanced, because attacks are rarer.
- Split by whole hour blocks: 104,912 train, 44,096 test windows, so no leakage between neighbouring seconds.

The dashboard

- Live status, event chart and alert log (MAC, severity, confidence)
- Replays a dataset or watches a live monitor-mode adapter

Results

99.34%

accuracy on 44,096 unseen test windows

Confusion matrix

	Predicted normal	Predicted attack
Actually normal	**38,904** Correctly ignored	**27** False alarms
Actually attack	**265** Missed attacks	**4,900** Attacks caught

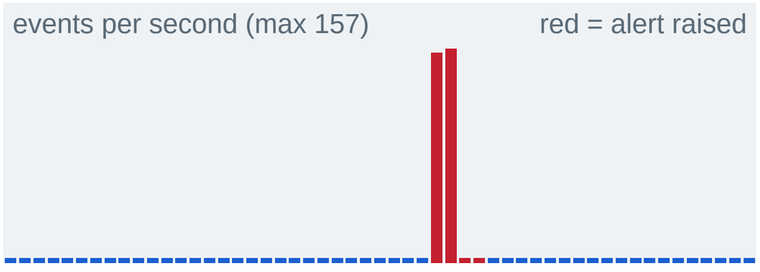
Recall (attacks caught) **94.9%**

Precision (alerts that were real) **99.5%**

What the model relies on

Events, last 60 s	32%
Events, last 10 s	27%
Time since last event	25%
Events this second	7%
Shortest gap	6%

Detection in a simulated attack



Replay of demo_attack.csv: normal traffic, a flood, then normal again. The system raised alerts on the flood seconds only.

Limitations

- Attack and normal captures cover different dates, so the model may partly learn each device's habits.
- Most missed attacks are single-event seconds that look normal on their own.
- Only deauthentication and disassociation floods are covered.
- A spoofed flood shows under the access point's MAC, not the attacker's.

What I would improve

- Add a WIPS step: log or block the offending MAC automatically.
- Test on my own live captures and report the difference.
- Detect more attacks: evil twin, beacon flood, disassociation variants.
- Add channel hopping and try other models such as Gradient Boosting.

Use of AI tools

I used Claude (Anthropic) as a coding assistant to draft the dashboard, feature extraction and training scripts and to debug setup errors. I chose the scope, data and features, ran and tested everything myself, reviewed every file and can explain each part of the code. All work used a public dataset or an isolated lab network I am authorised to test.

Code: github.com/BasileMakutano/Wireless-Intrusion-Detection-System

Built with Python, scikit-learn, scapy and Flask. Everything runs locally.

Dataset: association/disassociation event dataset provided for the course [add citation]